

Article

Iterative Audit Convergence in LLM-Managed Multi-Agent Systems: A Case Study in Prompt-Engineering Quality Assurance

Elias Calboreanu 

Swift AI Lab, The Swift Group, LLC, Reston, VA 20190, USA; ecalboreanu@theswiftgroup.com

Abstract

Prompt specifications for multi-agent large language model (LLM) systems carry data contracts and integration logic across interdependent files but are rarely subjected to structured-inspection rigor. We report a single-system case study of iterative, agent-driven auditing applied to AEGIS (Autonomous Engineering Governance and Intelligence System), a seven-lane production pipeline whose 7152-line specification surface was audited across nine rounds, surfacing 51 consistency defects (per-round counts of 15, 8, 12, 2, 8, 1, 4, 1, 0). We present a seven-category post hoc taxonomy with explicit coding rules, non-monotonic convergence consistent with cascading edits and audit-scope expansion, and a locked audit protocol. We further report two partial replications on a public synthetic mini-specification: a cross-LLM panel of four frontier vendors (OpenAI, Anthropic, Google, xAI; 12 traces; multi-vendor union detects all five seeded defects) and an inter-rater reliability check on a stratified subsample (Cohen's $\kappa = 0.80$ on category, 0.46 on severity). The full reproducibility bundle accompanies the submission.

Keywords: prompt engineering; multi-agent systems; iterative auditing; quality assurance; large language models; specification consistency; software inspection; cross-document validation; agentic automation; configuration management

1. Introduction

1.1. Motivation

Prompt engineering for multi-agent systems presents a quality assurance challenge distinct from traditional software testing. When an LLM-orchestrated pipeline spans multiple autonomous lanes, each with its own behavioral specification, data contracts, and integration points, the prompt layer becomes the system's source of truth. Errors in prompt specifications can propagate as runtime failures: wrong field names may break data handoffs, stale references can trigger invalid API calls, and missing labels can corrupt downstream classification. The empirical evidence presented in this paper is consistent with such failure modes but does not establish their relative frequency.

However, prompt specifications are rarely subjected to the rigor applied to source code. They are authored in natural language, reviewed informally, and updated incrementally. When a design change touches one document, ripple effects across other documents are easy to miss. Software inspection research has long established that structured review catches defects that testing alone may miss [1–3]. The challenge specific to multi-agent LLM systems is that the inspector is most naturally an LLM as well: human reviewers do not scale to seven interlinked lane specifications updated weekly, while LLM-driven walkthroughs can. That choice introduces a same-LLM-family concern—an inspector from

arXiv:2605.12280v2 [cs.SE] 19 Jun 2026



Academic Editors: Tadashi Dohi,
Junjun Zheng and Xiao-Yi Zhang

Received: 13 May 2026

Revised: 9 June 2026

Accepted: 16 June 2026

Published: 18 June 2026

Copyright: © 2026 by the author.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

the same model family as the author can share blind spots—which structures the design of this study and is taken up directly in the cross-LLM partial replication of Section 4.5.

Objective: The single main objective of this paper is to characterize, on one production system, what iterative LLM-driven auditing of a prompt-specification surface looks like in practice: which defect classes surface only across iterations, how convergence behaves, and what protocol parameters matter for replication. We pair the case study with two partial replications on a public synthetic mini-specification—a cross-LLM panel of four frontier vendors and an inter-rater reliability check—to bound the most-flagged threats to validity (a single LLM family auditing the system and single-coder taxonomy) before any generalization is attempted.

1.2. Research Questions

The research questions are stated descriptively rather than causally:

- **RQ1.** What defect classes remained after the initial audit pass and surfaced only through later iterative expanded-scope rounds in this case study?
- **RQ2.** What convergence behavior was observed, and how many rounds were required for this 7152-line, eight-document specification surface?

1.3. Contributions

This paper offers four contributions, each scoped to the case study and its partial replications:

- **C1 (taxonomy).** A seven-category post hoc taxonomy of the 51 surfaced defects, with category definitions, coding rules, and one representative example per category (Section 4 and Appendix B). Percentages are descriptive; no inferential claims are made.
- **C2 (observed convergence).** Per-round defect counts with Wilson 95% confidence intervals, a Mann–Kendall trend statistic on the convergence series, and a category-by-round cross-tabulation showing observed non-monotonic intermediate behavior consistent with cascading edits and audit-protocol maturation. We do not claim that the curve is predictable in advance; we report what was observed under explicitly stated confounds.
- **C3 (audit protocol distilled from the study).** An audit protocol distilled from the case study, with the final locked checklist, prompt template, finding schema, convergence criterion, and context-loading chronology released as a reproducibility appendix (Appendices A and C). We distinguish *protocol used during the study*, which co-evolved with discovery, from *protocol recommended after the study*.
- **C4 (partial replications bounding the central threats).** On a released public synthetic mini-specification, a four-vendor cross-LLM audit panel (12 traces from OpenAI, Anthropic, Google, and xAI; Section 4.5) and an inter-rater reliability check on a stratified 18-defect subsample (Cohen’s $\kappa = 0.80$ on category, 0.46 on severity; Section 4.6). Both are partial replications, not multi-system or human-IRR validations, and are scoped accordingly.

We do not claim that single-pass full-scope review is universally insufficient—only that single-file review cannot, by construction, detect cross-file defects, and that one-pass termination would have left later-discovered defects unresolved under this study’s protocol. A single-pass full-scope control was not run.

2. Background and Related Work

2.1. Software Inspection and Structured Review

Fagan-style code inspection [1] established that structured review detects defects that testing alone may miss. Boehm and Basili's "Software Defect Reduction Top 10 List" [2] reaffirmed this in 2001 with broader empirical evidence, identifying peer reviews and inspections as cost-effective defect-reduction mechanisms across multiple project contexts. Weinberg and Freedman [3] articulated lighter-weight walkthroughs and reviews, observing that effectiveness depends on checklist quality and reviewer expertise without requiring the full Fagan moderator/reader/recorder/inspector role separation.

Our audit protocol adapts the *walkthrough* tradition rather than full Fagan inspection. We did not have multiple human inspectors with separate role assignments; we had one LLM family acting as the inspector against a checklist authored by the same family. The locked checklist in Appendix A serves as the inspection guide, the sub-agent serves as the inspector, and the convergence criterion replaces the single-pass exit gate. The key limitation, that the inspector belongs to the same LLM family that authored the specifications, is documented in Section 5.5.

2.2. Specification Drift, Aging, and Traceability

Parnas [4] identified software aging as the phenomenon by which documentation accuracy degrades as the system that it describes evolves. Gotel and Finkelstein [5] analyzed the requirements traceability problem, finding that cross-artifact consistency is the most labor-intensive and error-prone aspect of requirements management. Our findings are consistent with both observations in the prompt-specification domain: 12 of the 51 defects (23.5%) were stale Jira references and 5 (9.8%) were cross-lane schema mismatches.

2.3. LLM Self-Evaluation and Self-Refinement

Three nearby lines of work all operate *within* an LLM's context window; our case study operates *across* sessions on persistent file-based artifacts. Self-consistency [6] samples multiple reasoning paths and selects the most consistent answer to a single prompt. Self-refinement [7] iterates a critique-and-revise loop on a single completion within one LLM. Lubos et al. [8] evaluated LLMs as requirements-QA checkers against ISO/IEC/IEEE 29148 [9]-style quality characteristics applied to one requirement at a time. Each of these methods asks the following question: *Is this completion or this requirement consistent with itself or with a rubric, right now?* The present study asks a different question: *Across many persistent files and multiple sessions, can iterative audit-and-fix drive a specification surface to quiescence at all, and what does the convergence path look like?* The persistence of file-based state changes the convergence dynamics: drift can accumulate across rounds, fixes can cascade into new defects, and a single inspector's blind spots persist across iterations unless cross-vendor evidence is introduced. Patil [10] and Akshathala [11] made the parallel point for LLM-based software quality assurance: final-pass success on its own is insufficient.

2.4. Recent LLM-Based SE and Agentic Quality Assurance

He, Treude, and Lo [12] surveyed LLM-based multi-agent systems across the software development lifecycle and identified robustness and synergy gaps. Cemri et al. [13] proposed a 14-failure-mode taxonomy for multi-agent LLM systems with human-annotated inter-rater agreement ($\kappa = 0.88$); their work directly motivates Appendix B's coding rules. Lubos et al. [8] evaluated LLMs for requirements quality assurance against ISO 29148-style requirement-quality characteristics.

Chen et al. [14] argued that prompts are software artifacts requiring lifecycle methods (the *promptware* position). Patil [10] and the agentic-AI assessment-framework literature [11]

are consistent with the position that final-pass success on its own is an inadequate quality signal. Repository-level agentic code reasoning benchmarking [15] provides a parallel for cross-file consistency challenges. Yang et al.'s SWE-agent [16] establishes a leading framework for LLM-driven single-agent repository interaction; AgentBench [17] provides standardized evaluation of LLMs-as-agents in synthetic environments. The present case study complements both lines of work by reporting production-deployment evidence rather than benchmark performance. MetaGPT [18] operationalizes standard operating procedures for multi-agent collaboration; AEGIS's seven-lane structure is a domain-specialized instance of that pattern. Naqvi, Baqar, and Mohammad [19] contrasted iterative closed-loop multi-agent testing against static single-shot test generation.

2.5. Companion Work

The companion preprint [20] addresses the runtime behavior of the same AEGIS pipeline, including STRIDE-based adversarial testing and FMEA-based safety analysis. The 51 specification defects reported here are distinct from the 51 STRIDE-categorized adversarial code findings reported in the companion: those are P0–P3 priority code vulnerabilities (lock handling, race conditions, subprocess management) found by a two-person human review panel across three injection rounds; ours are spec-consistency defects (version drift, stale Jira refs, schema mismatches) found by Claude sub-agents across nine audit rounds. The numerical coincidence is genuine but the populations are distinct in source, method, and content.

This paper exercises the prompt-specification layer that configures AEGIS; it does not validate the upstream Context Engineering [21], MANDATE [22], LATTICE [23], or TRACE [24] frameworks themselves. Those frameworks provide system context only.

3. Materials and Methods

3.1. System Under Test: AEGIS

AEGIS (Autonomous Engineering Governance and Intelligence System) is a seven-lane agentic automation pipeline operating against a Jira backlog that had grown to 2100+ tickets as of April 2026, up from the companion paper's approximately 1602-row consolidated backlog snapshot reported in [20]. Each lane is a Claude agent governed by a `PROMPT.md` file that defines the following:

- Behavioral rules and constraints;
- Data schemas for inter-lane communication;
- Model Context Protocol (MCP) [25] tool permissions and rate limits;
- Jira transition contracts (status IDs, label conventions);
- Worker pool configuration (parallelism, isolation);
- Quality gates.

The seven lanes share a common `TICKET_CONTRACT.md` that codifies status-transition IDs, creation authority by lane, mandatory labels, and shared schemas. The lanes communicate via JSON intermediary files (for example, Lane 2 writes findings to `lane_02_untracked_findings.json`; Lane 3 ingests them and creates Jira tickets). A design change in any one lane can invalidate assumptions in three to four other documents.

Representativeness of the Released Synthetic Mini-Specification: Reviewers ask how representative the synthetic fixture released in the reproducibility bundle (Appendix C) is of the real AEGIS surface. We answer with aggregate descriptors of the production specs (Table 1), all of which are computed directly from the released anonymized defect catalog or from line- and reference-counts on the production spec files; file-level specification content and Jira surface text are not reproduced in the Supplementary Materials because the aggregate descriptors are sufficient for the representativeness comparison, and because

the underlying files include third-party data covered by ongoing contractual constraints. The fixture preserves the property that the audit method targets—cross-document reference density—at the same order of magnitude (real AEGIS 12.77 references per 100 lines; fixture 10.20) while simplifying the scale by roughly thirty-fold (8 files vs. 4; median 926 lines per file vs. 30). It is a faithful scaled-down reproduction of the cross-document structure that makes cross-document auditing necessary, appropriate for replicating the audit method (as Section 4.5 does on this exact fixture), but not a scale model of the production system; absolute defect counts and rounds-to-convergence should not be extrapolated from the fixture to AEGIS. Defect density is intentionally higher in the fixture (5 seeded defects in 114 lines) than in production (per-category rates of 0.58–1.74 per 1000 lines) so that a reviewer can exercise every checklist dimension on a tractable artifact.

Table 1. Aggregate descriptors of the production AEGIS specification surface and the released synthetic mini-specification fixture. AEGIS values computed on the production specs as of 9 June 2026 and on the released 51-row anonymized defect catalog. Per-category defect rates use the seven-category taxonomy of Section 4.2.

Descriptor	Real AEGIS	Synthetic Mini-Spec
File count	8	4
Median lines per file (IQR)	926 (696–1098)	30 (24–32)
Cross-reference density (refs/100 lines)	12.77	10.20
Distinct schema-element count	76	5
Per-category defect rate (per 1000 lines)	version_drift 1.30; stale_jira_ref 1.74; cross_lane_schema 0.72; missing_lane_coverage 1.01; label_contract 0.87; semantic_text 1.16; formula_timing 0.58	5 seeded defects in 114 lines
Mean files involved per defect	1.27 (computed from the released catalog’s <code>files_involved</code> column)	1.40 (computed analogously on the five seeded defects)
Cross-file defect rate (% of defects spanning > 1 file)	27% (14 of 51)	40% (2 of 5)
Per-file defect distribution (count per file, sorted)	12, 12, 11, 9, 9, 5, 4, 3 (sum 65 file-references over 51 defects across 8 files)	2, 1, 1, 1, 0 across 4 files; 2 cross-file defects accounting for the surplus

Schema-element count is the distinct backticked field/key identifiers named across all spec files (a broader scope than the four shared JSON contract schemas listed in Table 2, which count only inter-lane contract objects).

Table 2. Specification surface metrics for the AEGIS pipeline at the time of the audit. Line counts as reported in the companion paper [20] (Section 6.1).

Metric	Value
Configuration files audited	8 (seven lane PROMPT.md files plus one shared TICKET_CONTRACT.md)
Lane PROMPT.md files	7 (one per lane: Lanes 1–7)
Total lane PROMPT lines	6907
Ticket contract lines	245
Total specification lines	~7152
Cross-document references	37 identified (cataloged by grep + manual confirmation across the eight files)
Shared data schemas	4 (fix_queue, untracked_findings, connectivity, fallback)
Shared transition IDs	4 (To Do, In Progress, On Hold, Done)
Ticket-creating lanes	4

AEGIS is also governed by separately documented Context Engineering [21], MAN-DATE [22], LATTICE [23], and TRACE [24] specifications. These frameworks provide system context, but the present study evaluates the consistency of the prompt-specification layer rather than validating the frameworks themselves. Specification-surface metrics are summarized in Table 2.

3.2. Case Study Design

We follow the case-study reporting conventions of Runeson and Höst [26].

Case Selection Rationale: AEGIS was selected because (a) it is a production-deployed multi-agent system whose specification surface had grown organically across lane consolidations and architectural changes, (b) the author had artifact-level access required to run the audit, and (c) the system had reached sufficient operational maturity for cross-document audit to surface meaningful defects.

Unit of Analysis: The unit of analysis is the prompt-specification surface (eight files) and the audit process (nine rounds). Convergence is measured at the audit-round level; defect classification is measured at the individual-finding level.

Propositions: RQ1 implies the proposition that *defect classes exist that single-file review cannot detect by construction*. RQ2 implies the proposition that *iterative auditing exhibits non-monotonic convergence in this case*.

Chain of Evidence: Audit logs (per-round finding lists with file/line citations) are the primary data; the AEGIS specification files are the substrate; the author's classification is the interpretation layer. The reproducibility bundle (Appendix C) preserves the audit logs and classification labels.

3.3. Audit Protocol (As Used in the Study)

Each audit round consisted of spawning one or more Claude sub-agents (via the Agent tool) with a checklist covering seven dimensions:

1. Version consistency within and across files;
2. Cross-lane data contract alignment (field names, schemas);
3. Jira permission boundaries (which lanes may create, comment, and transition);
4. Label conventions per the ticket contract;
5. Lane-count propagation (all references must cite seven lanes);
6. Cadence and scheduler alignment;
7. Internal contradictions between sections of the same document.

Agents read each file top-to-bottom and returned structured findings with exact line numbers. Fixes were applied between rounds using targeted edits, and verification greps confirmed each fix. Following Weinberg and Freedman [3], we adopted *walkthrough* discipline rather than full Fagan inspection: there was one inspector (the LLM family) rather than the moderator/reader/recorder/inspector role separation that Fagan inspection requires. Section 5.5 documents the limitations that this entails.

The audit protocol co-evolved with the fixes. Early rounds used per-file agents; later rounds explicitly loaded producer and consumer specifications together for cross-lane checks. Section 3.6 documents this chronology. Consequently, the convergence curve in Section 4 reflects two interleaved processes—defect remediation and audit refinement—and should be read as describing the joint process rather than as evidence about a fixed audit function. The final locked protocol used after the study is reproduced in Appendix A; that locked version is the one that we recommend for replication.

3.4. Defect Coding

The seven-category taxonomy reported in Section 4.2 was derived post hoc by a single coder (the author) and is therefore subject to single-coder bias. This is foregrounded as a central methodological limitation, not a peripheral one. Cemri et al. [13] achieved $\kappa = 0.88$ inter-rater agreement on a comparable taxonomy by using multiple coders; the present study did not. Future replications should run a quick inter-rater check on a defect subsample.

Category definitions, decision rules for boundary cases, a worked boundary case, and one representative example per category are provided in Appendix B. Categories were treated as mutually exclusive at coding time; any defect that could plausibly fit two categories was assigned to the one matching the *trigger of detection*, not the apparent fix.

3.5. Convergence Criterion (Operational, Not Mathematical)

The audit loop terminated when a full-scope audit (all eight files, all seven checklist dimensions) returned zero findings on round 9. Operationally, the loop resembles audit-to-quiescence: repeat audit-and-fix until a full-scope pass returns no new findings. This is analogous to fixed-point iteration, but the analogy is methodological rather than mathematical: the audit function changes across rounds (Section 3.3 caveat), the LLM auditor is stochastic, and a single clean pass is not a mathematical fixed point of a stationary operator.

A stricter protocol would require *two consecutive clean passes* to guard against oscillation. We did not adopt that stricter criterion in this study. The justification for the one-clean-pass rule actually used is operational: the audit was undertaken against a production system on a finite timeline, and the cost of an additional full-scope round was substantial; we elected to stop at round 9's clean pass and defer the stricter criterion to replication. *We recommend the two-consecutive-clean-pass rule as the replication criterion*, and we specify it formally in Appendix A.3. This recommendation is foregrounded here rather than buried in the Appendix, because the stopping rule is a known source of premature-termination risk on stochastic LLM auditors [6].

3.6. Context-Loading Chronology

The audit protocol used during the study evolved as documented in Table 3. The nine rounds were conducted across approximately five weeks (early April through early May 2026), after Lane 7 was introduced on 31 March 2026 per the companion paper [20]. The specification surface was held stable across audit rounds, modulo the targeted fixes applied between rounds; no architectural changes were introduced during the audit window.

Table 3. Context-loading chronology for the nine audit rounds.

Round	Files Loaded per Call	Cross-Lane Comparison?	Notes
1	1	No	Per-file structural pass
2	1	No	Per-file structural pass
3	1–2	Partial	First cross-lane checks for label conventions
4	2–3	Yes	First explicit producer-consumer schema comparison
5	2–3	Yes	Stabilized cross-lane comparison strategy
6	8	Yes	First full-scope pass
7	8	Yes	Full-scope, including changelog and example data
8	8	Yes	Full-scope; surfaced <code>priority_score</code> vs. <code>fix_priority</code> mismatch
9	8	Yes	Clean pass; no findings

4. Results

4.1. Convergence Data

Table 4 presents the per-round convergence data. The audit surfaced 51 total prompt-specification defects, distinct from the 51 STRIDE-categorized adversarial code findings reported in [20] (Table 4 of the companion). The per-round counts were 15, 8, 12, 2, 8, 1, 4, 1, and 0. Figure 1 shows the convergence curve with per-round discovery rates and cumulative totals.

Table 4. Audit convergence across nine rounds. Per-round shares are reported with Wilson 95% confidence intervals computed on the share of the total $n = 51$ defects surfaced in each round. CIs are reported as a descriptive uncertainty band on each round's contribution to the total, not as inference about an underlying defect-generating distribution; round-to-round counts are not independent draws (see Section 4.3).

Round	Issues Found	Share	95% CI (Wilson)	Primary Category	Cumulative
1	15	0.294	[0.187, 0.430]	Structural: stale versions, removed features, missing lanes	15
2	8	0.157	[0.082, 0.280]	Operational: fallback references, footer versions, lock commands	23
3	12	0.235	[0.140, 0.368]	Semantic: misleading env-var docs, Jira templates, cadence references	35
4	2	0.039	[0.011, 0.132]	Schema: timing assumptions, <code>fix_queue</code> field alignment	37
5	8	0.157	[0.082, 0.280]	Contract: <code>fallback_enabled</code> flag, labels, status contract coverage	45
6	1	0.020	[0.003, 0.103]	Changelog: stale fallback queue reference in version history	46
7	4	0.078	[0.031, 0.185]	Cross-lane: formula mismatch, planned scripts, transition scope	50
8	1	0.020	[0.003, 0.103]	Data contract: <code>priority_score</code> vs. <code>fix_priority</code> field-name break	51
9	0	0.000	[0.000, 0.070]	Clean pass: zero issues across all eight files	51

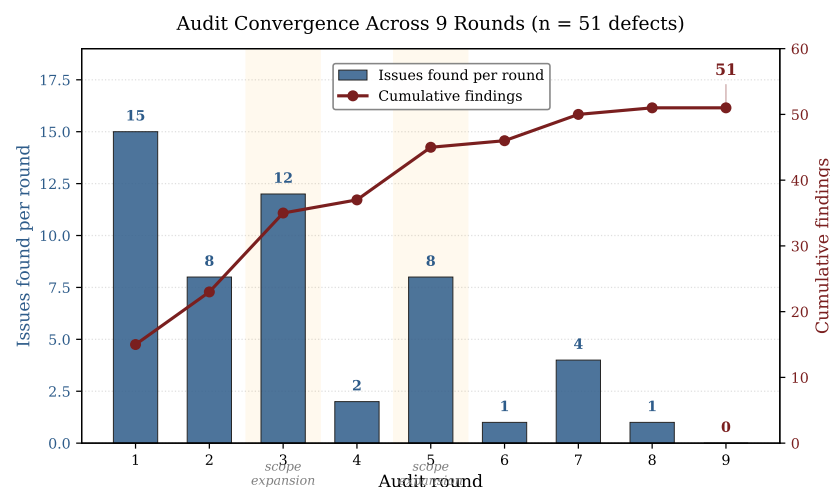


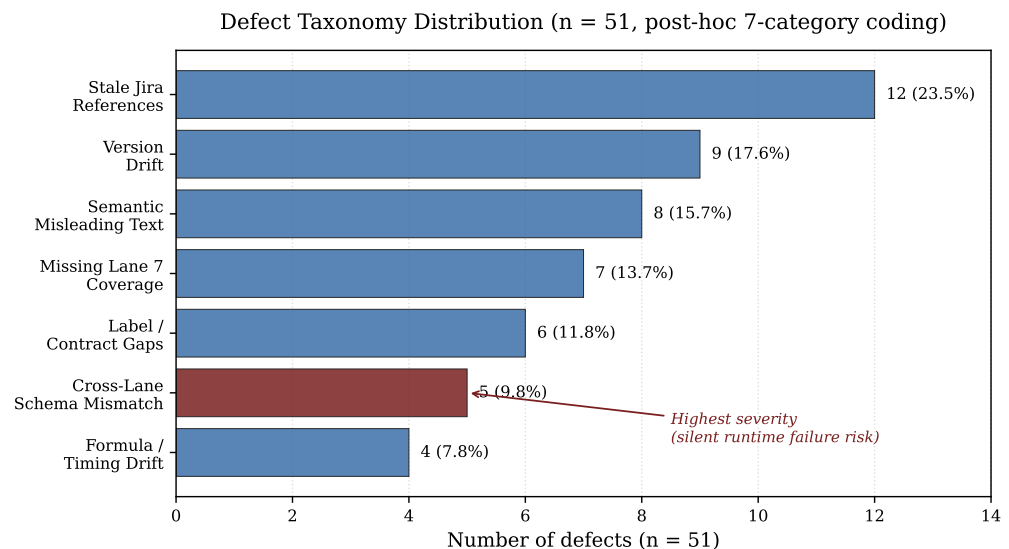
Figure 1. Audit convergence across nine rounds. Bars show per-round defect counts (left axis); the line shows cumulative totals (right axis). Shaded regions in rounds 3 and 5 indicate observed scope expansion.

4.2. Defect Taxonomy

Table 5 presents the post hoc seven-category taxonomy; Figure 2 shows the same distribution as a horizontal bar chart with the highest-severity category highlighted. Category definitions, coding rules, and examples are given in Appendix B. Stale Jira references (23.5%) were the most prevalent category, reflecting ripple effects of an earlier architectural change that removed Jira ticket creation from Lane 2. Cross-lane schema mismatches (9.8%), though fewer in count, were author-coded as high-severity because such mismatches can cause silent runtime failures (Section 4.3 and Appendix A).

Table 5. Defect classification (n = 51). Percentages sum to 99.9 due to rounding.

Defect Type	n	%
Version drift	9	17.6
Stale Jira references	12	23.5
Cross-lane schema mismatch	5	9.8
Missing Lane 7 coverage (recent extension)	7	13.7
Label/contract gaps	6	11.8
Semantic misleading text	8	15.7
Formula/timing drift	4	7.8

**Figure 2.** Defect taxonomy distribution (n = 51). Cross-lane schema mismatches (highlighted) are the highest-severity category by author coding.

Severity Distribution (author-coded per Appendix A.4 rubric): High severity, 5 (9.8%); medium, 32 (62.7%); low, 14 (27.5%). The high-severity category and the cross-lane schema mismatch category are coextensive in this study: every cross-lane schema mismatch was coded as high-severity per Appendix A.4’s silent-runtime-failure criterion, and no other defect class reached the high-severity threshold.

Objective Severity Proxies: The author-coded high/medium/low rubric is supplemented with a lightweight objective metric per category, as reported in Table 6. These are descriptive surface measurements drawn directly from the released defect catalog and the synthetic mini-spec, not replacements for the rubric. Their purpose is to give a reviewer a category-specific quantity that does not depend on author judgment; the inter-rater reliability check in Section 4.6 reports $\kappa = 0.46$ for severity, which is independent evidence that severity grading is the most judgment-dependent axis of the taxonomy and worth supplementing in this way.

Table 6. Objective severity proxies per taxonomy category. Each proxy is a surface-measurable count that captures one dimension of the defect’s blast radius, supplementing (not replacing) the author-coded H/M/L rubric.

Category	Proxy
version_drift	Number of distinct file pairs in which version strings disagree
stale_jira_ref	Number of stale field-name or behavior occurrences referenced
cross_lane_schema	Number of producer–consumer file pairs whose schema fields disagree
missing_lane_coverage	Number of cross-document references that omit one or more lanes
label_contract	Number of label-string variants used in place of the contract-canonical form
semantic_text	Number of within-file body/changelog pairs whose claims contradict
formula_timing	Magnitude of the timing or formula deviation, scaled to the documented value

Defect Density: Fifty-one defects across 7152 specification lines yielded approximately 7.1 defects per thousand lines of specification. This provides an order-of-magnitude reference point, but direct comparison to code-inspection rates from the Fagan-tradition empirical literature [1,2] is not warranted without comparable inspection-effort normalization: the artifact class (natural-language prompt specifications versus source code), the inspector (LLM versus human), the defect definition, and the inspection depth all differ.

4.3. Key Observations

Observed Non-Monotonic Convergence: Rounds 3 and 5 surfaced more issues than round 2. We interpret this as the audit scope expanding rather than a regression: structural fixes in earlier rounds revealed previously masked inconsistencies. Auditor-variance vs. mechanism interpretation is discussed in Section 5.5.3.

Two Intertwined Signals—Defect-Removal Convergence and Audit-Protocol Maturation: The per-round series in Figure 1 blends two processes that the case-study design cannot separate. The first is *defect-removal convergence*: as fixes accumulate between rounds, fewer defects remain to be found, which is the convergence behavior that the protocol is designed to drive. The second is *audit-protocol maturation*: as documented in the context-loading chronology (Table 3), the inspector’s effective coverage expanded from single-file scope in rounds 1–2, to partial cross-file scope in rounds 3–5, to full-scope cross-document loading from round 6 onward. The bumps at rounds 3 and 5 are therefore consistent with two non-exclusive explanations: structural fixes from earlier rounds unmasking previously hidden inconsistencies (the cascading-edits interpretation of Section 5.2), and the audit prompt itself becoming capable of finding defect classes that it could not previously see. We interpret the non-monotonic pattern as the joint signature of both processes rather than as an intrinsic property of any one audit method, and the formal trend statistic reported below is conditioned on this confound. The locked Appendix A protocol—with the full seven-dimension checklist and explicit full-scope context loading from round 1—is the version recommended for replicators precisely so that future runs see only the first signal, not the second.

Cross-Lane Defects Are the Hardest to Catch with Single-File Review: The most operationally consequential defect (round 8) was a pre-remediation field-name mismatch: Lane 3 emitted `priority_score` while Lane 4 was specified to consume `fix_priority`. This could plausibly have caused a silent runtime failure, such as empty sort results, with no error signal. The defect was remediated before the companion paper’s final architecture review, and both lanes now use `priority_score` (companion paper [20], Section 3.2). By construction, single-file review cannot detect this class of defect. A full-scope single-pass review *might* detect it; we did not run such a control.

Specification Aging Outlasts Structural Fixes: After all structural issues were resolved, rounds 6 through 8 continued to surface semantic inconsistencies. These were not detected by the grep-based verification used in this study. Parnas [4] identified this phenomenon as software aging; the prompt-specification analog observed here is *specification aging*.

Trend Statistic for the Convergence Series. A nonparametric Mann–Kendall trend test on the per-round count vector (15, 8, 12, 2, 8, 1, 4, 1, 0) yields $S = -26$, $Z = -2.64$ (with continuity correction; tie-corrected variance $\text{Var}(S) = 90$ from two tied pairs), and a two-tailed $p = 0.008$, with Kendall's $\tau = -0.72$. The series shows a statistically significant decreasing trend by this test. We report this descriptively and treat the p -value as approximate for two reasons: rounds are not independent draws from a fixed audit function (the protocol matured across rounds as documented above, and removed defects do not regenerate), and the population of detectable defects is bounded by the specification's finite content. The trend statistic is consistent with the cascading-edits and audit-protocol-maturation interpretation, but it does not by itself rule out alternatives; see Section 5.5.3.

4.4. Defect Category Progression

Figure 3 cross-tabulates defect categories against audit rounds; Table 7 provides full numeric detail. Three patterns are visible: First, version drift and stale Jira references concentrate in rounds 1–3 (19 of 21 combined, or 90.5%), consistent with structural issues surfacing earliest. Second, semantic misleading text peaks in round 3 (5 of 8 total). Third, cross-lane schema mismatches appear exclusively in rounds 4–8 (all five occurrences), consistent with the observation that this class requires multi-file visibility, which (Table 3) only became standard from round 4.

Table 7. Defect cross-tabulation: category by round.

Defect Type	R1	R2	R3	R4	R5	R6	R7	R8	R9	Total
Version drift	5	3	0	0	1	0	0	0	0	9
Stale Jira refs	5	3	3	0	1	0	0	0	0	12
Cross-lane schema	0	0	0	1	1	0	2	1	0	5
Missing Lane 7	4	0	2	0	1	0	0	0	0	7
Label/contract	1	1	2	0	2	0	0	0	0	6
Semantic text	0	0	5	0	2	0	1	0	0	8
Formula/timing	0	1	0	1	0	1	1	0	0	4
Column total	15	8	12	2	8	1	4	1	0	51

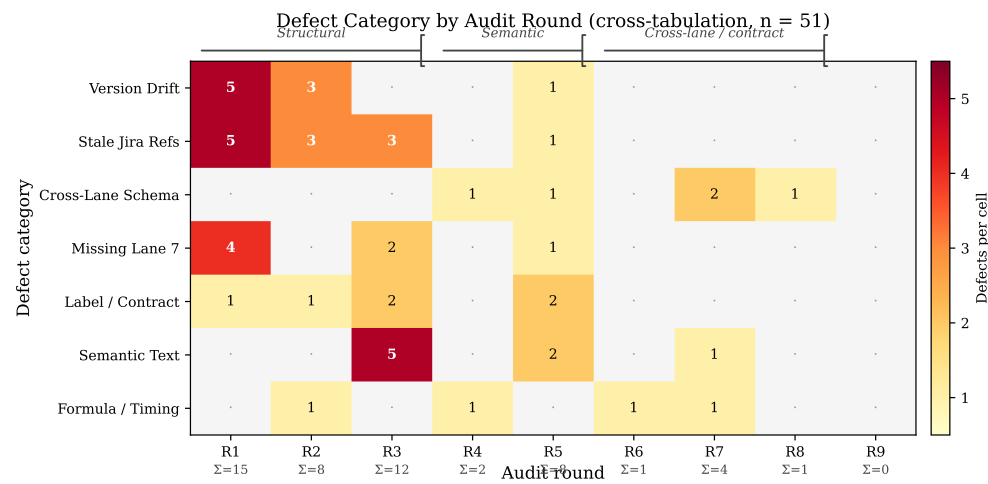


Figure 3. Defect category by audit round (cross-tabulation, $n = 51$). Phase brackets above the heatmap mark three observed phases of the convergence path: a *structural* phase (rounds 1–2) dominated by version drift and stale Jira references, a *semantic* phase (round 3) when misleading-text defects peak, and a *cross-lane/contract* phase (rounds 4–8) when multi-file context loading first becomes standard and the cross-lane schema and label-contract defects surface. These bracket labels are descriptive of the observed pattern in this case study, not pre-registered phase definitions.

4.5. Cross-LLM Partial Replication on the Synthetic Mini-Specification

To address the same-LLM-family concern directly—and on a fixture that every reader can re-run—we applied the locked seven-dimension audit protocol (Appendix A) to the released synthetic mini-specification (Appendix C): a four-file, ~114-line synthetic intake-triage pipeline (named ATLAS in the released fixture) with five seeded defects spanning five taxonomy categories and known ground truth. Each of four frontier vendors (Anthropic, OpenAI, Google, xAI) received an identical, blinded copy of the mini-spec—with all ground-truth annotations and seeded-defect category names removed—together with the locked checklist and severity rubric, and they returned a structured finding list under a fixed JSON schema. The gold-label key was withheld from every model and used only for scoring. Each vendor was run three times to characterize within-model variance, yielding 12 audit traces in total. Models were called at temperature 0 where supported; models that do not expose a temperature control (the GPT-5.x and Claude-4.x families) were run at their default and the value recorded as such. Exact model identifiers, decoding settings, token counts, and wall-clock times are recorded in the released run traces (Appendix C).

Per-Vendor Detection: Table 8 reports, per vendor and averaged over three runs, how many of the five seeded defects were detected, the spurious-finding count, and agreement with the gold labels on category and severity for detected defects. A defect is scored as *detected* when at least one finding cites the correct file(s) and identifies the underlying issue; following the original study’s trigger-of-detection convention, a correct detection under a different category label still counts as a detection, with the category disagreement tracked separately.

Table 8. Per-vendor detection on the synthetic mini-specification (mean of three runs per vendor).

Vendor	Model	Detected/5	Spurious	Cat. Agmt	Sev. Agmt
OpenAI	gpt-5.5-2026-04-23	5.00	0.67	1.00	1.00
Anthropic	claude-opus-4-8	4.67	1.33	0.85	0.57
Google	gemini-3.1-pro-preview	5.00	0.00	1.00	1.00
xAI	grok-4.3	4.00	0.00	1.00	1.00

The central result is unambiguous: frontier models that had no access to Claude’s authoring process and no ground-truth key independently recovered the seeded defects at high rates. OpenAI’s GPT-5.5 and Google’s Gemini 3.1 Pro each detected all five seeded defects in every run; the re-baselined Claude detected all five in two of three runs; xAI’s Grok detected four of five in every run. Spurious findings were rare (0–1.3 per run on average) and, where they occurred, generally flagged a real but unseeded consistency issue rather than a hallucinated one. This is direct evidence against the shared-blind-spot hypothesis for this fixture: three independent vendors recovered essentially the same defect set that the Claude-family auditor did.

Per-Defect Detection: Table 9 gives the per-defect detection rates across the panel.

Table 9. Per-seeded-defect detection rate (fraction of three runs in which the defect was detected, per vendor).

ID	Category	Sev.	Scope	OpenAI	Claude	Gemini	Grok	Overall
S1	version_drift	low	single	1.00	1.00	1.00	1.00	1.00
S2	stale_jira_ref	medium	single	1.00	1.00	1.00	0.00	0.75
S3	cross_lane_schema	high	cross	1.00	0.67	1.00	1.00	0.92
S4	label_contract	medium	cross	1.00	1.00	1.00	1.00	1.00
S5	semantic_text	medium	single	1.00	1.00	1.00	1.00	1.00

The two defects that any model missed were the two that this manuscript’s argument predicts are hardest. The cross-lane, highest-severity schema break (S3) was caught by three of four vendors in every run; only Claude missed it, and only in one of three runs—the run-to-run variance that the three-run protocol (Appendix A) exists to absorb. The stale-Jira reference (S2) was caught by three vendors every time but missed by Grok in all three runs—a clean, reproducible, vendor-specific blind spot. The two blind spots are vendor-specific and non-overlapping, which is itself the case for a multi-vendor panel: no single model is uniformly best, but the union of the panel detects every defect. A pilot of the Gemini leg on the smaller gemini-2.5-flash model missed S3 in all runs, so detection of the cross-lane defect also scales with model capability—a useful bound for replicators choosing models.

Cross-Vendor Agreement: Table 10 reports pairwise agreement between vendors on which seeded defects were *confirmed* detected (in ≥ 2 of 3 runs): the Jaccard overlap of confirmed-detected sets, and Cohen’s κ on the category labels that the two vendors assigned to the defects they both detected.

Table 10. Pairwise cross-vendor agreement on confirmed detections.

Vendor A	Vendor B	Jaccard	Cohen’s κ (Categories)
OpenAI	Claude	1.00	0.76
OpenAI	Gemini	1.00	1.00
OpenAI	Grok	0.80	1.00
Claude	Gemini	1.00	0.76
Claude	Grok	0.80	0.69
Gemini	Grok	0.80	1.00

OpenAI, Claude, and Gemini confirm-detected the identical set of all five defects (Jaccard = 1.00 among them); Grok’s 0.80 Jaccard against the other three reflects exactly one missing defect (S2), not category disagreement. On the defects that the vendors co-detected, category agreement is high to perfect ($\kappa = 0.69$ –1.00), with OpenAI/Gemini/Grok agreeing perfectly pairwise; the only κ below 0.76 involves Claude’s one divergent category label on the sort-order defect (S5). In short, the panel disagrees on at most one borderline category label and one vendor’s single coverage gap—not on what the defects are.

Scope: This is a partial replication, bounded in three specific ways. First, it is not a multi-system replication: running the protocol on additional independent specification surfaces remains future work, and the cross-vendor agreement reported here does not bound the same-LLM-family risk on the production AEGIS specs, only on the released fixture. Second, it is not a single-pass full-scope control comparison; the panel does not compare iterative auditing against a one-pass alternative, only against itself across vendors. Third, it is not a human-only or hybrid human-LLM comparison; no human reviewer audited the mini-spec. What the panel does provide—and the specific scope under which it should be cited—is direct empirical evidence that non-Claude frontier models recover the same seeded defects under the locked protocol on a public fixture, which is the specific evidence that the shared-blind-spot concern calls for. Figure 4 visualizes this as a 5×4 detection matrix: 18 of 20 cells at 1.00, and the two coverage gaps that exist are vendor-specific and non-overlapping. All twelve run traces, the gold-label key (released after the v15 review cycle to preserve the held-out replication value), and the scoring scripts can be found in the Supplementary Materials.

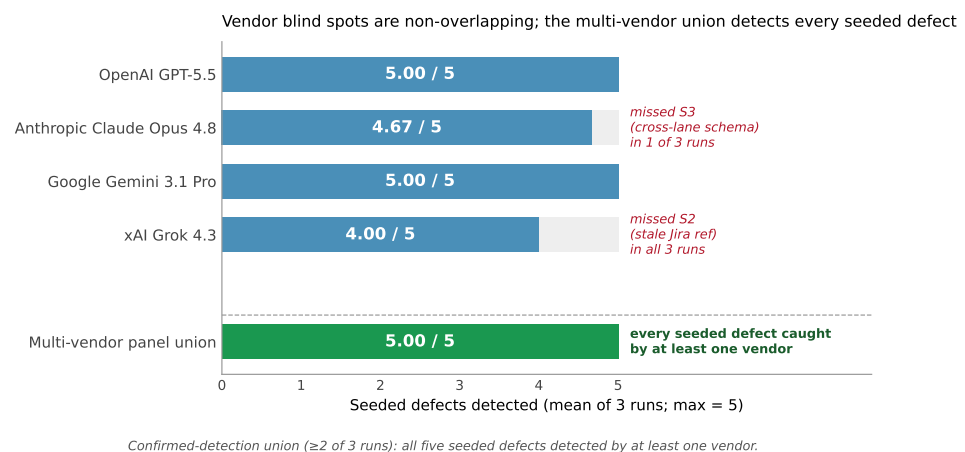


Figure 4. Per-vendor detection of the five seeded defects on the synthetic mini-spec (mean of three runs each), with the multi-vendor panel union shown separately at the bottom. The two coverage gaps that exist are vendor-specific and non-overlapping (Grok missed the stale-Jira reference S2 in all three runs; Claude missed the cross-lane schema break S3 in one of three runs); the panel union therefore detects every seeded defect. This is direct evidence against the shared-LLM-family-blind-spot hypothesis on this fixture: if the concern were operative, the non-Claude vendors should miss the same defects that Claude misses, which they do not.

4.6. Inter-Rater Reliability: Partial Replication

The seven-category taxonomy and severity grading reported above were developed and applied by a single coder (the author). Following the inter-rater reliability conventions of empirical software engineering [13]—noting that Cemri et al. [13]’s reported $\kappa = 0.88$ benchmark is on a different 14-category multi-agent-failure taxonomy with multiple human coders, so it serves as a reference point for the methodology rather than a direct comparison—we report a partial replication on a stratified subsample of the released 51-defect catalog (Appendix C).

Design: We drew a stratified random subsample of 18 of the 51 cataloged defects (reproducible; fixed random seed 42), constrained to include at least one defect from each of the seven taxonomy categories. A second coder then re-coded each defect’s category and severity from the locked taxonomy rules and severity rubric alone, blind to the original author labels (the second coder received only the redacted defect description and the files involved). We report Cohen’s κ between the original author labels and the second-coder

labels, on category and on severity, each with a nonparametric bootstrap 95% confidence interval (5000 resamples of the 18-defect subsample), along with raw percent agreement.

Second Coder: For the second coder, we used the same four-vendor LLM panel as in Section 4.5; each model independently coded the subsample from descriptions alone, and the modal label across the panel served as the second coder. We frame this honestly as multi-LLM coder agreement on a held-out subsample—evidence that the taxonomy categories are model-distinguishable from descriptions alone—and not as a substitute for human inter-rater reliability. A human second-coder replication is the natural next step and is supported by the same instrument; the pipeline supports re-running the analysis with a human coder’s CSV in place of the LLM-panel modal labels.

Results: Table 11 reports the agreement statistics.

Table 11. Inter-rater reliability on the 18-defect stratified subsample. Second coder: four-LLM panel modal consensus. The four panel coder identifiers (each model called once per defect to produce a label; the modal label across the four served as the second coder) are OpenAI gpt-5.5-2026-04-23, Anthropic claude-opus-4-8, Google gemini-2.5-flash, and xAI grok-4.3. The Gemini coder ran at flash tier for this from-description categorization task because category-from-description is less capability-sensitive than the Section 4.5 detection audit, which used gemini-3.1-pro-preview; all four model identifiers are recorded per coding decision in the released panel-coding detail (irr_panel_detail.json).

Metric	Value	95% CI	Landis–Koch Band
Cohen’s κ —category	0.801	[0.564, 1.000]	almost perfect
Cohen’s κ —severity	0.458	[0.008, 0.811]	moderate
% agreement—category	0.833	—	raw agreement
% agreement—severity	0.722	—	raw agreement

Category agreement is substantial-to-almost-perfect ($\kappa = 0.80$), indicating that the seven categories are applied consistently from descriptions alone across four independent vendors. Severity agreement is moderate ($\kappa = 0.46$), but the 95% confidence interval [0.008, 0.811] spans from near-chance to almost-perfect: the lower bound approaches the “slight” Landis–Koch band, so the moderate point estimate should be read as the central tendency of a wide interval rather than as a precise reliability claim. This is consistent with severity grading being the most judgment-dependent dimension of the taxonomy, and it is the empirical motivation for the objective severity proxies in Table 6. The severity κ is unweighted (nominal); a linear-weighted κ that credits adjacent-level near-misses (e.g., medium $\leftrightarrow$ high) would be higher.

Disagreement Audit: The category disagreements are interpretable rather than arbitrary, and they cluster on one taxonomy boundary. All three on this subsample re-coded the author’s label as `semantic_text`: two `formula_timing` defects (a hard-stop budget quoted inconsistently in an operator-facing example, three of four coders; a changelog entry referencing a removed file, two of four) and one `stale_jira_ref` defect (a relocated “fallback queue” diagnostic, on which all four coders agreed). Each turns on a trigger-of-detection judgment—which checklist dimension surfaced the defect—rather than a category being indefensible, and the recurring drift into `semantic_text` (internal contradiction) is a concrete, actionable signal for sharpening the `formula_timing` and `stale_jira_ref` definitions against it.

Scope: This is a partial replication on a subsample, with an LLM-panel second coder; it bounds, rather than eliminates, the single-coder threat. The taxonomy claim is refined from “post hoc, single-coded” to “post hoc, single-coded with κ -validated partial replication.” The subsample, both coders’ labels, the κ computation, and the bootstrap CIs are included in the Supplementary Materials for independent recomputation.

5. Discussion

5.1. Three Distinct Notions of Review

“Single-pass review” is ambiguous. We use three distinct terms:

- **Single-File Review:** The auditor reads one specification file at a time, without explicit cross-file comparison. By construction, single-file review cannot detect cross-file schema mismatches.
- **Initial Full-Scope Pass:** The auditor reads all files in one context window, once, with cross-file comparison enabled. We did not run this as a control.
- **Iterative Full-Scope Auditing:** Repeated full-scope passes with audit-and-fix between rounds, until quiescence. This is the protocol distilled from the case study.

The defensible empirical claim of this paper is that, in this system, single-file review missed cross-document defect classes by construction, and one-and-done audit (any single-pass design) left later-discovered defects unresolved under the case-study protocol. We do not claim that no full-scope single-pass design could in principle have surfaced them.

5.2. Cascading Edits

Fixing issue A in round N can create a new inconsistency B that was masked by A. For example, removing `fallback_enabled: true` from Lane 2’s config (round 5) made the changelog entry referencing `jira_fallback_queue.md` newly incorrect (caught in round 6). This is analogous to regression introduction in traditional code review.

5.3. Cross-Document Visibility

No single-file audit can detect that Lane 4 expects `fix_priority` while Lane 3 emits `priority_score`. This requires reading both files in the same pass and comparing schemas field-by-field. The audit protocol itself was iteratively refined to support this, as documented in Table 3; convergence in this case study therefore depends on both fixing defects and improving the audit prompt.

5.4. Implications for Prompt Engineering Practice (Case-Bounded)

The following are observations from this single case study under the locked protocol, not generalizations to other systems, scales, or LLM families. Each item is framed as *what we observed in this system*, not as a normative prescription. Replication across diverse architectures with dissimilar models is required before any of the points below can be treated as practice recommendations.

- **Treat Prompt Specifications as Code, in this System:** The behavior that we observed—prompt files carrying data contracts and integration logic, and breaking them silently when a single document was edited in isolation—is consistent with the promptware position [14]. Whether this generalizes to LLM-orchestrated pipelines of different scales or architectures is an open question.
- **Iterative Auditing Budget at this Scale and Protocol:** In this case, the locked protocol required nine rounds on a 7152-line, eight-document surface to reach a clean pass; the cross-document defects in rounds 7–8 would have been missed by earlier rounds. The metric that we used was a full-scope clean pass, not a fixed round count. Rounds-to-convergence at other scales is unknown.
- **Cross-Document Audits Caught Contract Defects Single-File Review Could Not, in this Study:** By construction, single-file review cannot detect cross-lane data-contract issues; in this study every cross-lane defect surfaced only after multi-file context loading was enabled. A full-scope single-pass review might or might not detect them; we did not run such a control.

- **Verification Greps Were Necessary but Not Sufficient, in this Case:** Automated string matching caught structural issues but missed semantic drift on this system’s specification surface.

The cross-LLM partial replication in Section 4.5 addresses the case-bounded character of these claims from one direction—different vendors recover the same seeded defects under the locked protocol on a public fixture—but does not address multi-*system* generalization, which remains an open topic for future work. We also note explicitly here what the cross-LLM panel did *not* provide: a single-pass full-scope control comparison and a human-only or hybrid human–LLM review. The single-pass control would have required running an additional cohort of full-scope audits on the same fixture under a different stopping rule; the human-only and hybrid comparisons would have required coder recruitment outside the revision window. Each remains the natural next study; the cross-LLM panel is the alternative that we could execute on the released public fixture in this revision cycle.

Operational Outcomes Are Out-of-Scope and Reported Elsewhere: This paper characterizes the audit process and its convergence behavior; it does not measure whether correcting the 51 surfaced defects improved system performance, reliability, or operational outcomes. Post-fix runtime behavior of the same AEGIS pipeline (STRIDE-categorized adversarial code findings, lock-handling and race-condition fixes, FMEA safety analysis) is reported in the companion preprint [20]; the 51 specification defects audited here are distinct in source, method, and content from the 51 code findings reported there. A full operational-validation study that ties specific spec-fix events to downstream runtime metrics is the natural follow-on and is left for future work.

5.5. Threats to Validity

We identified four categories of threats following Runeson and Höst [26] and Wohlin et al. [27]. The first two—shared LLM blind spots and single-coder taxonomy—were the most-flagged concerns in peer review and are foregrounded here; both are now partially bounded by the v15 evidence in Sections 4.5 and 4.6, but not eliminated by it.

5.5.1. Same-LLM-Family Auditing and Single-Coder Taxonomy

Two related limitations sit at the foundation of this case study and motivated the v15 partial replications.

Same LLM Family Writes and Audits: The same LLM family (Claude) both authored AEGIS’s prompt specifications and ran the audit; the inspector and the author could in principle share blind spots, and a clean round-9 pass demonstrates only internal consistency under that single family’s vantage point, not ground-truth correctness. The cross-LLM panel in Section 4.5 bounds this concern empirically on the released fixture: three independent vendors (OpenAI, Google, xAI) recovered the same seeded defects under the locked protocol, and the panel’s blind spots are vendor-specific and non-overlapping, so the multi-vendor union detected every defect. This is partial replication on a public fixture, not a multi-vendor audit of the production AEGIS specs themselves; the same-family risk for AEGIS proper is reduced, not eliminated.

Single-Coder Taxonomy: The seven-category taxonomy was developed and applied post hoc by a single coder (the author). Section 4.6 reports an inter-rater reliability partial replication on a stratified 18-defect subsample with a four-LLM panel as a modal second coder: Cohen’s κ on category is 0.80 (almost perfect), while on severity it is 0.46 (moderate). Category agreement is high, and severity less so—consistent with severity being the more judgment-dependent axis. We frame the LLM-panel coder honestly as multi-LLM coder agreement on a held-out subsample, not as a substitute for human inter-rater reli-

ability; a human second coder remains the strongest replication and is supported by the same instrument.

The taxonomy claim in this paper is therefore positioned as a post hoc categorization with κ -validated partial replication, not as a fully validated empirical-SE taxonomy.

5.5.2. External Validity and Representativeness

This study examines a single system ($n = 1$) without a control group; the seven-category taxonomy is post hoc and may not be exhaustive. The implication is direct: claims in Section 5 should be read as case-specific. The representativeness analysis in Section 3 compares aggregate descriptors of the production AEGIS surface against the released synthetic mini-spec (Table 1); the fixture preserves cross-reference density at the same order of magnitude while simplifying scale by roughly thirty-fold, which licenses replicating the audit *method* but not extrapolating absolute counts. Multi-system replication on independent specification surfaces remains the strongest answer and is left for future work.

5.5.3. Construct and Statistical Validity

Construct: Convergence to zero findings does not establish correctness, only internal consistency under the locked protocol's coverage. Runtime validation remains a necessary complementary quality gate; we report nothing about runtime behaviour here.

Statistical and Conclusion Validity: The sample sizes are modest ($n = 9$ rounds, $n = 51$ defects). The Mann–Kendall trend test reported in Section 4.3 is significant ($p = 0.008$), but the rounds are not independent draws (audit-scope expansion is the explicit confound, and removed defects do not regenerate). With a stochastic LLM auditor and no repeat trials per round, the bumps at rounds 3 and 5 cannot be statistically distinguished from auditor variance without a control comparison; the cascading-edits interpretation is one explanation among several. Following Wohlin et al. [27], conclusion validity concerns the treatment–outcome relationship: with one stochastic auditor, a co-evolving protocol, and no repeat-trial design, alternative explanations for the observed convergence pattern cannot be ruled out. Future replications should fix the protocol before remediation begins and run repeat-trial passes per round.

5.5.4. Audit-Protocol Co-Evolution and Sequence Effects

The audit protocol co-evolved with the discovered defects (Sections 3.3 and 4.3). Later prompts may have become tuned to the known defect space, weakening the convergence interpretation. Related, later rounds operated on artifacts already changed by prior fixes, so the order in which fixes were applied may have shaped the observed category progression: a different fix order might have surfaced different categories at different rounds. The locked Appendix A protocol was finalized after the study and should be applied as-is in future replications to address both confounds; the recommended two-consecutive-clean-pass stopping rule, promoted from the Appendix into the main text (Section 3, Convergence Criterion), is one such hardening.

6. Conclusions

This paper presents a single-system empirical case study of iterative, agent-driven auditing of prompt specifications in a production seven-lane orchestration pipeline. Across nine rounds, 51 prompt-specification defects were surfaced and remediated. The audit terminated when round 9 returned zero findings under the case-study protocol's one-clean-pass stopping rule; we recommend a stricter two-consecutive-clean-pass criterion for replication.

We make four case-bounded claims: First, in this case study, single-file review missed cross-document defect classes by construction. Second, the observed convergence path

was non-monotonic, consistent with cascading edits and audit-scope expansion; a Mann–Kendall trend test on the per-round counts was significant ($p = 0.008$, Kendall's $\tau = -0.72$) but conditioned on the round-non-independence confound, so we report the test descriptively. Third, the audit protocol itself co-evolved with the fixes; the locked, recommended version is reproduced in Appendix A. Fourth, two partial replications on a released public synthetic mini-specification bound the most-flagged threats to validity: a four-vendor cross-LLM panel (12 traces; OpenAI, Anthropic, Google, and xAI) detected the seeded defects independently of Claude, with the multi-vendor union recovering all five; and an inter-rater reliability check on a stratified 18-defect subsample of the catalog returned Cohen's $\kappa = 0.80$ on category (almost perfect) and $\kappa = 0.46$ on severity (moderate).

Iterative full-scope auditing is a candidate practice for LLM-orchestrated systems where prompt specifications serve as the behavioral contract between autonomous agents. The v15 partial replications bound the same-LLM-family and single-coder threats on a public fixture but do not address multi-*system* replication; the latter remains required, alongside a single-pass full-scope control and a human second coder, to validate generalizability.

Supplementary Materials: The following supporting information can be downloaded at <https://www.mdpi.com/article/10.3390/software5020026/s1>: the reproducibility bundle described in the Data Availability Statement, comprising per-round defect counts (`round_counts.csv`), the anonymized 51-row defect catalog (`defect_catalog.csv`), the locked audit checklist, prompt template, and severity rubric, the context-loading chronology, the synthetic mini-specification with five seeded defects, the twelve cross-LLM audit traces with detection and agreement CSVs, the inter-rater reliability subsample and Cohen's κ computation, the aggregate AEGIS representativeness descriptors, and a `README.md` with replication instructions.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable. This study did not involve human or animal subjects.

Informed Consent Statement: Not applicable.

Data Availability Statement: A reproducibility bundle accompanies this submission as Supplementary Materials containing the following: per-round CSVs; the anonymized 51-row defect catalog with category, severity, files-involved, and dimension metadata per defect; the locked checklist, prompt template, and severity rubric; the context-loading chronology; the synthetic mini-specification for protocol validation; the 12 cross-LLM audit traces (Section 4.5) with detection/agreement CSVs; the inter-rater reliability subsample, panel detail, and κ computation (Section 4.6); aggregate AEGIS representativeness descriptors (Table 1); and the multi-vendor evaluation pipeline used to produce the new evidence. The gold-label key for the mini-specification is held back from public release until after the v15 review cycle to preserve the fixture's held-out replication value; reviewers may request it from the authors. The full AEGIS specification files themselves and the Jira backlog text are not included in the Supplementary Materials because the aggregate descriptors in Table 1 are sufficient for the representativeness comparison, and because the underlying files include third-party data covered by ongoing contractual constraints with customers of The Swift Group. The bundle is sufficient for full replication of the audit method against any independent specification surface.

Acknowledgments: During the preparation of this study, the author used Anthropic's Claude (model versions: Claude Opus and Claude Sonnet, active across March–May 2026) as the inspector-agent in the case-study protocol described in the Materials and Methods section and the audit protocol Appendix. The use of these large language models is the substantive empirical subject and methodology of the study, rather than a writing-assistance tool. The author has reviewed all model-generated outputs, performed all defect coding, constructed the taxonomy, and takes full responsibility for the content of this publication. No generative AI tool was used for superficial text editing beyond the documented methodology.

Conflicts of Interest: The author is employed by The Swift Group, LLC, which holds commercial licensing rights to products built on the AEGIS reference implementation. The audit method and data presented here are released independently of any commercial product. The employer had no role in the study design, the audit protocol design, the analysis, the interpretation, or the decision to submit.

Abbreviations

The following abbreviations and notations are used in this manuscript:

AEGIS	Autonomous Engineering Governance and Intelligence System
ATLAS	Synthetic four-lane intake-triage pipeline used as the released mini-spec
CI	Confidence interval
FMEA	Failure Mode and Effects Analysis
IQR	Interquartile range
IRR	Inter-rater reliability
JSON	JavaScript Object Notation
κ	Cohen's kappa (chance-corrected inter-rater agreement)
LLM	Large language model
MCP	Model Context Protocol
MLT	MANDATE/LATTICE/TRACE governance stack
QA	Quality assurance
RQ	Research question
SE	Software engineering
STRIDE	Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
S	Mann–Kendall statistic (sum of signed pairwise comparisons)
τ	Kendall's tau (rank-correlation effect size)
Z	Standard normal test statistic (Mann–Kendall)

Appendix A. Locked Audit Checklist, Prompt Template, and Method Specifics

Appendix A.1. Locked Checklist (Seven Dimensions)

The full locked checklist is reproduced in the Supplementary File `audit_checklist_locked.md`. The seven dimensions are version consistency, cross-lane data contract alignment, Jira permission boundaries, label conventions, lane-count propagation, cadence and scheduler alignment, and internal contradiction.

Appendix A.2. Locked Prompt Template

The full locked prompt template is reproduced in the Supplementary File `audit_prompt_template.md`. It specifies the structured-output schema (id, file, line, dimension, category, severity, description, suggested_fix, uncertainty), the trigger-of-detection rule for category assignment, and the severity rubric.

Appendix A.3. Recommended Convergence Criterion

Terminate when two consecutive full-scope passes return zero findings. This is stricter than the criterion used in the case study, which required only one clean pass.

Appendix A.4. Severity Rubric

The full severity rubric is reproduced in the Supplementary File `severity_rubric.md`. High severity covers silent runtime failure risk and producer–consumer schema breaks; medium covers documented-behavior errors and operator-misleading text; low covers cosmetic and internal-doc drift.

Appendix A.5. Method Specifics for LLM-Based Auditing

For replication, report the following: LLM family (Claude, this study), specific model versions (mix of Claude Opus and Sonnet across nine rounds), decoding parameters (default temperature; no custom sampling), context-window policy (single-file rounds 1–2, expanding to multi-file rounds 3–5, full-scope rounds 6–9), sub-agents per round (1–3), and tooling (Claude Agent tool with file-read access; no cached memories; fresh session per round). A more rigorous replication would lock the model version and decoding parameters before remediation begins.

Appendix B. Defect Coding Rules and Examples

Coder: Single coder (the author). Inter-rater validation was not performed; this is acknowledged as a central limitation in Section 5.5.

Mutual Exclusivity: Categories are mutually exclusive. Boundary cases were assigned to the category matching the trigger of detection, not the apparent fix.

Worked Boundary Case: A defect surfaced in round 5 read “Lane 5’s diagnostic checklist instructs the operator to confirm that the `fallback_enabled` flag is set to true if Lane 2 is in degraded mode, but Lane 2’s config file has `fallback_enabled: false` after the prior architectural change.” This defect could plausibly be coded as either `semantic_text` (the diagnostic instruction has become misleading) or `label_contract` (the flag-value contract has drifted). The trigger of detection was a label-contract dimension check (dimension 4), so it was coded `label_contract`.

The seven category definitions and one example per category are reproduced in the Supplementary File `audit_checklist_locked.md` and the manuscript’s full mark-down source.

Appendix C. Reproducibility Artifact

The reproducibility bundle (Supplementary Materials) contains the following: `round_counts.csv`, `defect_catalog.csv` (anonymized 51-row catalog with category, severity, files-involved, and dimension per defect), `audit_checklist_locked.md`, `audit_prompt_template.md`, `severity_rubric.md`, `context_loading_chronology.csv`, `synthetic_mini_specification.md` (a four-file mini-specification with five seeded defects across five taxonomy categories for protocol validation against ground truth), `aegis_aggregate_descriptors.md` (the aggregate descriptors reported in Table 1), and a `README.md` with replication instructions. The full AEGIS specification files and Jira backlog are not included because the aggregate descriptors and the released catalog are sufficient for the representativeness analysis, and because the files contain customer-attached third-party data; the bundle is sufficient to replicate the audit *method* against any independent specification surface.

References

1. Fagan, M.E. Design and Code Inspections to Reduce Errors in Program Development. *IBM Syst. J.* **1976**, *15*, 182–211.
2. Boehm, B.; Basili, V.R. Software Defect Reduction Top 10 List. *Computer* **2001**, *34*, 135–137.
3. Weinberg, G.M.; Freedman, D.P. Reviews, Walkthroughs, and Inspections. *IEEE Trans. Softw. Eng.* **1984**, *SE-10*, 68–72.
4. Parnas, D.L. Software Aging. In Proceedings of the 16th International Conference on Software Engineering (ICSE), Sorrento, Italy, 16–21 May 1994; pp. 279–287.
5. Gotel, O.C.Z.; Finkelstein, A.C.W. An Analysis of the Requirements Traceability Problem. In Proceedings of the 1st International Conference on Requirements Engineering (ICRE), Colorado Springs, CO, USA, 18–21 April 1994; pp. 94–101.
6. Wang, X.; Wei, J.; Schuurmans, D.; Le, Q.; Chi, E.; Zhou, D. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In Proceedings of the International Conference on Learning Representations (ICLR), Kigali, Rwanda, 1–5 May 2023.
7. Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; et al. Self-Refine: Iterative Refinement with Self-Feedback. In Proceedings of the Advances in Neural Information Processing Systems (NeurIPS), New Orleans, LA, USA, 10–16 December 2023.
8. Lubos, S.; Felfernig, A.; Tran, T.N.T.; Garber, D.; El Mansi, M.; Polat Erdeniz, S.; Le, V.M. Leveraging LLMs for the Quality Assurance of Software Requirements. *arXiv* **2024**, arXiv:2408.10886. <https://doi.org/10.48550/arXiv.2408.10886>.
9. ISO/IEC/IEEE 29148:2018. Systems and Software Engineering—Life Cycle Processes—Requirements Engineering; International Organization for Standardization (ISO): Geneva, Switzerland, 2018.
10. Patil, A. A Blueprint for AI-Driven Software Quality: Integrating LLMs with Established Standards. *arXiv* **2025**, arXiv:2505.13766. <https://doi.org/10.48550/arXiv.2505.13766>.
11. Akshathala, S.; Adnan, B.; Ramesh, M.; Vaidhyanathan, K.; Muhammed, B.; Parthasarathy, K. Beyond Task Completion: An Assessment Framework for Evaluating Agentic AI Systems. *arXiv* **2025**, arXiv:2512.12791. <https://doi.org/10.48550/arXiv.2512.12791>.
12. He, J.; Treude, C.; Lo, D. LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead. *arXiv* **2024**, arXiv:2404.04834. <https://doi.org/10.48550/arXiv.2404.04834>.
13. Cemri, M.; Pan, M.Z.; Yang, S.; Agrawal, L.A.; Chopra, B.; Tiwari, R.; Keutzer, K.; Parameswaran, A.; Klein, D.; Ramchandran, K.; et al. Why Do Multi-Agent LLM Systems Fail? *arXiv* **2025**, arXiv:2503.13657. <https://doi.org/10.48550/arXiv.2503.13657>.
14. Chen, Z.; Wang, C.; Sun, W.; Liu, X.; Zhang, J.M.; Liu, Y. Promptware Engineering: Software Engineering for Prompt-Enabled Systems. *arXiv* **2025**, arXiv:2503.02400. <https://doi.org/10.48550/arXiv.2503.02400>.
15. Li, J.; Su, Y.; Lyu, M.R. From Laboratory to Real-World Applications: Benchmarking Agentic Code Reasoning at the Repository Level. *arXiv* **2026**, arXiv:2601.03731. <https://doi.org/10.48550/arXiv.2601.03731>.
16. Yang, J.; Jimenez, C.E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; Press, O. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. *arXiv* **2024**, arXiv:2405.15793. <https://doi.org/10.48550/arXiv.2405.15793>.
17. Liu, X.; Yu, H.; Zhang, H.; Xu, Y.; Lei, X.; Lai, H.; Gu, Y.; Ding, H.; Men, K.; Yang, K.; et al. AgentBench: Evaluating LLMs as Agents. In Proceedings of the International Conference on Learning Representations (ICLR), Vienna, Austria, 7–11 May 2024.
18. Hong, S.; Zhuge, M.; Chen, J.; Zheng, X.; Cheng, Y.; Wang, J.; Zhang, C.; Wang, Z.; Yau, S.K.S.; Lin, Z.; et al. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. *arXiv* **2023**, arXiv:2308.00352. <https://doi.org/10.48550/arXiv.2308.00352>.
19. Naqvi, S.; Baqar, M.; Mohammad, N.A. The Rise of Agentic Testing: Multi-Agent Systems for Robust Software Quality Assurance. *arXiv* **2026**, arXiv:2601.02454. <https://doi.org/10.48550/arXiv.2601.02454>.
20. Calboreanu, E. Closed-Loop Autonomous Software Development via Jira-Integrated Backlog Orchestration: A Case Study in Deterministic Control and Safety-Constrained Automation. *arXiv* **2026**, arXiv:2604.05000. <https://doi.org/10.48550/arXiv.2604.05000>.
21. Calboreanu, E. *Context Engineering: A Methodology for Structured Human-AI Collaboration*; Working Paper v2.5; Capitol Technology University: Laurel, MD, USA, 2026.
22. Calboreanu, E. MANDATE: A Tolerance-Based Framework for Autonomous Agent Task Specification. 2026. SSRN Paper 6170328. Available online: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6170328 (accessed on 12 May 2026).
23. Calboreanu, E. LATTICE: A Governance-First Architecture for Authorized Autonomous AI Operations. 2026. SSRN Paper 6151128. Available online: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6151128 (accessed on 12 May 2026).
24. Calboreanu, E. TRACE: A Governance-First Execution Framework Providing Architectural Assurance for Autonomous AI Operations. 2026. SSRN Paper 6212818. Available online: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6212818 (accessed on 12 May 2026).
25. Anthropic. Model Context Protocol Specification. 2024. Available online: <https://modelcontextprotocol.io/> (accessed on 12 May 2026).

26. Runeson, P.; Höst, M. Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empir. Softw. Eng.* **2009**, *14*, 131–164.
27. Wohlin, C.; Runeson, P.; Höst, M.; Ohlsson, M.C.; Regnell, B.; Wesslén, A. *Experimentation in Software Engineering*; Springer: Berlin/Heidelberg, Germany, 2012.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.